Поздравляю! После небольшой правки обучение заработало:

```
8       class Program
9       {
10          static void Main(string[] args)
11          {
12              Console.WriteLine("=== СТАРТ ТЕСТИРОВАНИЯ ОПТИМИЗАТОРА ADAM ===");
13
14              // 1. Инициализируем контекст OpenCL (preferAccelerator: true для запуска на GPU)
15              using var computeCtx = new ComputeContext(preferAccelerator: true);
16
17              // Регистрируем очередь выполнения в потоке (чтобы работал DeviceQueueRegistry)
18              // Примечание: Убедись, что при создании контекста ты вызываешь метод регистрации,
19              // например: DeviceQueueRegistry.RegisterCurrent(computeCtx.CreateQueue());
20              // Для теста предполагаем, что очередь уже активна внутри ComputeContext.
21              var context = computeCtx.Context;
22              DeviceQueueRegistry.RegisterCurrent(context.CreateCommandQueue(computeCtx.Device));
23              // 2. Создаем обучаемые параметры (например, веса модели)
24              // Форма: [1, 2, 4] - 3D тензор, как ожидает твой Bmm (страница 24)
25              int[] shape = { 1, 2, 4 };
26
```

При запуске важно регестирироваать контекст чтобы все слои могли получить доступ к актуальной очереди команд!

=== СТАРТ ТЕСТИРОВАНИЯ ОПТИМИЗАТОРА ADAM ===
максимальные размеры локальной группы для устройства на основе варпа:X=256,Y=4
Could not open file: C:\Users\╨Ф╨Й╨╕ ╤В╤А╨╕ ╨╣\AppData\Local\Temp\dep-4bfb26.d
Начальные веса успешно загружены во VRAM.
Запуск цикла оптимизации (20 эпох)...

Could not open file: C:\Users\╨Ф╨Й╨╕ ╤В╤А╨╕ ╨╣\AppData\Local\Temp\dep-5b415f.d
Could not open file: C:\Users\╨Ф╨Й╨╕ ╤В╤А╨╕ ╨╣\AppData\Local\Temp\dep-2ecf6b.d
Could not open file: C:\Users\╨Ф╨Й╨╕ ╤В╤А╨╕ ╨╣\AppData\Local\Temp\dep-e58ef8.d
Could not open file: C:\Users\╨Ф╨Й╨╕ ╤В╤А╨╕ ╨╣\AppData\Local\Temp\dep-db4d50.d
Could not open file: C:\Users\╨Ф╨Й╨╕ ╤В╤А╨╕ ╨╣\AppData\Local\Temp\dep-0a8245.d
Could not open file: C:\Users\╨Ф╨Й╨╕ ╤В╤А╨╕ ╨╣\AppData\Local\Temp\dep-3a6bf8.d
Could not open file: C:\Users\╨Ф╨Й╨╕ ╤В╤А╨╕ ╨╣\AppData\Local\Temp\dep-915c26.d
Эпоха 01 | Текущий MSE Loss: 3,847500
Could not open file: C:\Users\╨Ф╨Й╨╕ ╤В╤А╨╕ ╨╣\AppData\Local\Temp\dep-962b56.d
Could not open file: C:\Users\╨Ф╨Й╨╕ ╤В╤А╨╕ ╨╣\AppData\Local\Temp\dep-1a27e8.d
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==========
Данные (Forward):
  [0,5000, -0,2000, 0,1000, 0,9000, 0,3000, 0,7000, -0,5000, 0,2000]

Градиенты (Backward):
  [-0,1250, -0,5500, -0,7250, -0,7750, 0,3250, 0,1750, -0,3750, -0,4500]
==========

Эпоха 02 | Текущий MSE Loss: 3,675000
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==========

Данные (Forward):
  [0,5500, -0,1500, 0,1500, 0,9500, 0,2500, 0,6500, -0,4500, 0,2500]

Градиенты (Backward):
  [-0,1125, -0,5375, -0,7125, -0,7625, 0,3125, 0,1625, -0,3625, -0,4375]
==

Эпоха 03 | Текущий MSE Loss: 3,507646
==
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==
Данные (Forward):
  [0,5998, -0,1000, 0,2000, 1,0000, 0,2001, 0,6001, -0,4001, 0,3000]

Градиенты (Backward):
  [-0,1001, -0,5250, -0,7000, -0,7500, 0,3000, 0,1500, -0,3500, -0,4250]
==

Эпоха 04 | Текущий MSE Loss: 3,345518
==
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==
Данные (Forward):
  [0,6492, -0,0501, 0,2499, 1,0499, 0,1502, 0,5505, -0,3502, 0,3498]

Градиенты (Backward):
  [-0,0877, -0,5125, -0,6875, -0,7375, 0,2876, 0,1376, -0,3375, -0,4125]
==

Эпоха 05 | Текущий MSE Loss: 3,188682
==
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==
Данные (Forward):
  [0,6980, -0,0003, 0,2998, 1,0998, 0,1005, 0,5012, -0,3005, 0,3996]

Градиенты (Backward):
  [-0,0755, -0,5001, -0,6751, -0,7251, 0,2751, 0,1253, -0,3251, -0,4001]
==

Эпоха 06 | Текущий MSE Loss: 3,037185
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==========
Данные (Forward):
  [0,7460, 0,0494, 0,3496, 1,1496, 0,0511, 0,4524, -0,2509, 0,4493]

Градиенты (Backward):
  [-0,0635, -0,4876, -0,6626, -0,7126, 0,2628, 0,1131, -0,3127, -0,3877]
==========

Эпоха 07 | Текущий MSE Loss: 2,891053
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==========
Данные (Forward):
  [0,7929, 0,0990, 0,3993, 1,1993, 0,0018, 0,4042, -0,2015, 0,4988]

Градиенты (Backward):
  [-0,0518, -0,4752, -0,6502, -0,7002, 0,2505, 0,1011, -0,3004, -0,3753]
==========

Эпоха 08 | Текущий MSE Loss: 2,750292
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==========
Данные (Forward):
  [0,8383, 0,1485, 0,4489, 1,2490, -0,0471, 0,3568, -0,1524, 0,5481]

Градиенты (Backward):
  [-0,0404, -0,4629, -0,6378, -0,6878, 0,2382, 0,0892, -0,2881, -0,3630]
==========

Эпоха 09 | Текущий MSE Loss: 2,614879
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)

```
============================================================
Данные (Forward):
  [0,8819, 0,1977, 0,4983, 1,2985, -0,0957, 0,3104, -0,1036, 0,5971]

Градиенты (Backward):
  [-0,0295, -0,4506, -0,6254, -0,6754, 0,2261, 0,0776, -0,2759, -0,3507]
============================================================


Эпоха 10 | Текущий MSE Loss: 2,484763
============================================================
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
============================================================
Данные (Forward):
  [0,9232, 0,2468, 0,5477, 1,3478, -0,1438, 0,2651, -0,0552, 0,6459]

Градиенты (Backward):
  [-0,0192, -0,4383, -0,6131, -0,6630, 0,2141, 0,0663, -0,2638, -0,3385]
============================================================


Эпоха 11 | Текущий MSE Loss: 2,359867
============================================================
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
============================================================
Данные (Forward):
  [0,9619, 0,2956, 0,5968, 1,3970, -0,1914, 0,2212, -0,0071, 0,6944]

Градиенты (Backward):
  [-0,0095, -0,4261, -0,6008, -0,6507, 0,2021, 0,0553, -0,2518, -0,3264]
============================================================


Эпоха 12 | Текущий MSE Loss: 2,240084
============================================================
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
============================================================
Данные (Forward):
  [0,9974, 0,3441, 0,6458, 1,4461, -0,2385, 0,1789, 0,0405, 0,7425]

Градиенты (Backward):
  [-0,0006, -0,4140, -0,5886, -0,6385, 0,1904, 0,0447, -0,2399, -0,3144]
============================================================
```

Эпоха 13 | Текущий MSE Loss: 2,125284
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==========
Данные (Forward):
  [1,0295, 0,3924, 0,6945, 1,4949, -0,2850, 0,1384, 0,0877, 0,7902]

Градиенты (Backward):
  [0,0074, -0,4019, -0,5764, -0,6263, 0,1787, 0,0346, -0,2281, -0,3024]
==========

Эпоха 14 | Текущий MSE Loss: 2,015315
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==========
Данные (Forward):
  [1,0576, 0,4403, 0,7430, 1,5436, -0,3308, 0,0999, 0,1342, 0,8376]

Градиенты (Backward):
  [0,0144, -0,3899, -0,5642, -0,6141, 0,1673, 0,0250, -0,2164, -0,2906]
==========

Эпоха 15 | Текущий MSE Loss: 1,910014
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==========
Данные (Forward):
  [1,0816, 0,4879, 0,7913, 1,5920, -0,3759, 0,0637, 0,1802, 0,8844]

Градиенты (Backward):
  [0,0204, -0,3780, -0,5522, -0,6020, 0,1560, 0,0159, -0,2049, -0,2789]
==========

Эпоха 16 | Текущий MSE Loss: 1,809211
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8

. Тип данных: System.Single (float32)
==========
Данные (Forward):
  [1,1012, 0,5351, 0,8393, 1,6401, -0,4201, 0,0300, 0,2255, 0,9308]

Градиенты (Backward):
  [0,0253, -0,3662, -0,5402, -0,5900, 0,1450, 0,0075, -0,1936, -0,2673]
==========

Эпоха 17 | Текущий MSE Loss: 1,712738
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==========
Данные (Forward):
  [1,1164, 0,5819, 0,8871, 1,6881, -0,4634, -0,0010, 0,2701, 0,9766]

Градиенты (Backward):
  [0,0291, -0,3545, -0,5282, -0,5780, 0,1341, -0,0002, -0,1825, -0,2558]
==========

Эпоха 18 | Текущий MSE Loss: 1,620434
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==========
Данные (Forward):
  [1,1272, 0,6283, 0,9345, 1,7357, -0,5058, -0,0291, 0,3140, 1,0219]

Градиенты (Backward):
  [0,0318, -0,3429, -0,5164, -0,5661, 0,1235, -0,0073, -0,1715, -0,2445]
==========

Эпоха 19 | Текущий MSE Loss: 1,532150
==========
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
==========
Данные (Forward):
  [1,1339, 0,6742, 0,9817, 1,7831, -0,5471, -0,0541, 0,3570, 1,0665]

Градиенты (Backward):
  [0,0335, -0,3315, -0,5046, -0,5542, 0,1132, -0,0135, -0,1608, -0,2334]

```
============================================================


Эпоха 20 | Текущий MSE Loss: 1,447747
============================================================
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
============================================================
Данные (Forward):
  [1,1365, 0,7196, 1,0284, 1,8301, -0,5873, -0,0761, 0,3991, 1,1105]

Градиенты (Backward):
  [0,0341, -0,3201, -0,4929, -0,5425, 0,1032, -0,0190, -0,1502, -0,2224]
============================================================


--------------------------------------------
Цикл завершен. Проверяем финальные веса на GPU...

Сравнение результатов (Target vs Trained):
Элемент [0]: Идеал = 1,00 | Обучено = 1,1356
Элемент [1]: Идеал = 2,00 | Обучено = 0,7645
Элемент [2]: Идеал = 3,00 | Обучено = 1,0749
Элемент [3]: Идеал = 4,00 | Обучено = 1,8768
Элемент [4]: Идеал = -1,00 | Обучено = -0,6264
Элемент [5]: Идеал = 0,00 | Обучено = -0,0948
Элемент [6]: Идеал = 1,00 | Обучено = 0,4403
Элемент [7]: Идеал = 2,00 | Обучено = 1,1537

=== ТЕСТ ЗАВЕРШЕН ===
```

Теперь переходим к реализации функций вактивации (ReLU,sigmoid и другие) а также исправлению компилятора.

Касаемо 1 проблемы:

🔍 Что удалось обнаружить в коде и логах

1. **Ошибка «Could not open file: ... dep-434ccf.d» в логах**
   - **Проблема:** На странице 4 лога видна классическая ошибка компилятора OpenCL. Она возникает, когда драйвер (особенно часто на встроенных картах Intel или некоторых версиях NVIDIA) пытается при сборке ядра создать временный файл зависимостей (`.d`) в системной директории `Temp` , но у процесса либо нет прав, либо директория заблокирована антивирусом.
   - **Решение:** При вызове program.Build `(...)` в классе `JITProgramCache` попробуй явно передать флаг сборки, отключающий генерацию дебаг-файлов зависимостей (например, строки опций `"-cl-no-program-source"` или специфичные флаги для подавления генерации `.d` файлов в зависимости от вендора).

Вот ссылка на исходный код бибилотеки для проверки, поддерживает ли она параметризацию https://github.com/Dima3108/NOpenCL/blob/87d5391bd3bc1cac48a6e887b06033a2693a3f39/NOpenCL/Program.cs#L111

```
15          /// <summary>
16          /// Возвращает скомпилированную программу из кэша или компилирует её «на лету», если её там нет.
17          /// </summary>
            Counter 15
18          public static Program GetOrCompile(Context context, Device device, string sourceCode)
19          {
20              lock (_lock)
21              {
22                  // В качестве ключа используем хэш-код исходного текста (или сам текст)
23                  if (_cache.TryGetValue(sourceCode, out var program))
24                  {
25                      return program;
26                  }
27  #if DEBUG
28                      File.WriteAllText("localkernel.cl", sourceCode);
29  #endif
30                  try
31                  {
32                      // Создаем и компилируем программу под наше устройство
33                      var newProgram = context.CreateProgramWithSource(sourceCode);
34                      newProgram.Build(new[] { device });
35
36                      _cache[sourceCode] = newProgram;
37
38                      return newProgram;
39                  }
40                  catch (Exception ex)
41                  {
42                      throw new InvalidOperationException("Ошибка JIT-компиляции OpenCL кода ядра:\n" + ex.Message, ex);
43                  }
44              }
45          }
46      }
47  }
48
```

Теперь переходим к проблеме номер 3 , вот актуальная версия кода:

3. **Отсутствие редукции при Broadcasting в обратном проходе (JIT)**
    - **Проблема:** Твой `LocalRoslynTranspiler` отлично считает смещения (strides) для прямого прохода с учетом правил Broadcasting (зануляя шаг, если размерность оси равна 1). Однако в обратном проходе (`fused_elementwise_backward_kernel` на странице 12) градиенты просто пишутся по индексу `idx_{name}`.
    - **Почему это баг:** Если тензор был "растянут" (например, `Bias` с формой `[1, 4]` прибавлялся к матрице `[2, 4]`), то в один и тот же индекс `idx_Bias` будут одновременно писать несколько потоков GPU. Происходит состояние гонки (Race Condition), и градиент затирается вместо того, чтобы суммироваться (аккумулироваться) по "растянутым" осям. В логах на странице 4 как раз видно: `Градиент весов Weight (первые 4): 0,00000`.
    - **Решение:** Для широковещательных тензоров в обратном проходе JIT-ядра необходимо использовать атомарное сложение `atomic_add` (если устройство поддерживает `atomic_xchg` для `float`) или делать операцию редукции (как ты уже сделал вручную на CPU для слоя `RMSNorm`).

```
using System;
using System.Collections.Generic;
using System.Linq.Expressions;
using System.Text;

namespace cstorchcl.compiler
{
    public class LocalExpressionVisitor : ExpressionVisitor
    {
        private readonly StringBuilder _bodyCode = new StringBuilder();

        // Отслеживаем только используемые тензоры для сигнатуры аргументов ядра
        public Dictionary<string, LocalVariable> UsedTensors { get; } = new
Dictionary<string, LocalVariable>();

        /// <summary>
        /// Генерирует оптимизированный исходный код OpenCL ядра с
подставленными константами и функциями дропаута.
        /// </summary>
        public string GenerateKernelSource()
        {
            var source = new StringBuilder();

            // Встраиваем ваш быстрый генератор случайных чисел и логику
дропаута в начало кода
            source.Append(@"// Встроенные утилиты генерации шума и Dropout
inline float nn_rand(int tid, unsigned int seed) {
```

```
    unsigned int x = tid + seed;
    x = (x ^ 61) ^ (x >> 16);
    x += (x << 3);
    x = x ^ (x >> 4);
    x *= 0x27d4eb2d;
    x = x ^ (x >> 15);
    return (float)x / 4294967296.0f;
}

inline float nn_dropout(float x, float p, int tid, unsigned int seed) {
    if (p <= 0.0f) return x;
    if (p >= 1.0f) return 0.0f;
    float r = nn_rand(tid, seed);
    if (r < p) {
        return 0.0f;
    } else {
        return x / (1.0f - p);
    }
}

");

            // Формируем сигнатуру ядра. Теперь здесь ТОЛЬКО буферы и их длины!
            source.Append("kernel void fused_elementwise_kernel(\n");

            int tensorIdx = 0;
            foreach (var kvp in UsedTensors)
            {
                source.Append($"    global float* {kvp.Key}, const int
{kvp.Key}_len");
                if (tensorIdx < UsedTensors.Count - 1) source.Append(",\n");
                tensorIdx++;
            }

            source.Append(") {\n");
            source.Append("    int tid = get_global_id(0);\n\n");

            // Оптимизация private-памяти (регистров GPU)
            source.Append("    // Чтение во внутренние регистры\n");
            foreach (var kvp in UsedTensors)
            {
                source.Append($"    float priv_{kvp.Key} = {kvp.Key}[min(tid,
{kvp.Key}_len - 1)];\n");
            }
            source.Append("\n");

            // Тело вычислений
            source.Append("    // Вычисления над регистрами\n");
            source.Append(_bodyCode.ToString());
            source.Append("\n");

            // Безопасное сохранение результатов во VRAM
            source.Append("    // Сохранение результатов во VRAM\n");
            foreach (var kvp in UsedTensors)
            {
                source.Append($"    if (tid < {kvp.Key}_len) {{ {kvp.Key}[tid] =
priv_{kvp.Key}; }}\n");
            }
            source.Append("}");

            return source.ToString();
        }
```

```
        protected override Expression VisitMethodCall(MethodCallExpression node)
        {
            // 1. Обработка метода .Assign(...) -> замена знака равенства '='
            if (node.Method.Name == "Assign" && node.Object is MemberExpression
leftVariable)
            {
                string varName = leftVariable.Member.Name;
                _bodyCode.Append($"    priv_{varName} = ");

                // Разбираем то, что передано внутрь Assign(...)
                Visit(node.Arguments[0]);
                _bodyCode.Append(";\n");
                return node;
            }

            // 2. Обработка стандартных математических функций активации
            if (node.Method.DeclaringType == typeof(nnclloc))
            {
                string funcName = node.Method.Name.ToLower();

                if (funcName == "dropout")
                {
                    _bodyCode.Append("nn_dropout(");
                    if (node.Arguments[0] is MemberExpression varMember)
                    {
                        _bodyCode.Append($"priv_{varMember.Member.Name}, ");
                    }
                    var pLambda =
Expression.Lambda<Func<float>>(node.Arguments[1]);
                    float pVal = pLambda.Compile()();

_bodyCode.Append(pVal.ToString(System.Globalization.CultureInfo.InvariantCulture
) + "f, tid, ");

                    var seedLambda =
Expression.Lambda<Func<uint>>(node.Arguments[2]);
                    uint seedVal = seedLambda.Compile()();
                    _bodyCode.Append(seedVal + "u)");
                }
                else
                {
                    _bodyCode.Append($"{funcName}(");
                    if (node.Arguments[0] is MemberExpression argMember)
                    {
                        _bodyCode.Append($"priv_{argMember.Member.Name}");
                    }
                    _bodyCode.Append(")");
                }
                return node;
            }

            return base.VisitMethodCall(node);
        }

        // Переписываем VisitBinary для обработки простых операций внутри Assign
(например, A / q)
        protected override Expression VisitBinary(BinaryExpression node)
        {
            Visit(node.Left);

            string opStr = node.NodeType switch
            {
                ExpressionType.Add => " + ",
```

```
                ExpressionType.Subtract => " - ",
                ExpressionType.Multiply => " * ",
                ExpressionType.Divide => " / ",
                _ => throw new NotSupportedException($"Операция {node.NodeType}
не поддерживается в JIT.")
            };

            _bodyCode.Append(opStr);
            Visit(node.Right);

            return node;
        }

        // Обновляем VisitMember, чтобы он правильно подставлял имена регистров
в правых частях уравнений
        protected override Expression VisitMember(MemberExpression node)
        {
            if (node.Type == typeof(LocalVariable))
            {
                string name = node.Member.Name;
                if (!UsedTensors.ContainsKey(name))
                {
                    var objectMember = Expression.Convert(node, typeof(object));
                    var getterLambda =
Expression.Lambda<Func<object>>(objectMember);
                    var localBuffer = (LocalVariable)getterLambda.Compile()();
                    UsedTensors.Add(name, localBuffer);
                }

                // Если мы уже внутри парсинга выражения, то подставляем имя
приватного регистра GPU
                _bodyCode.Append($"priv_{name}");
                return node;
            }

            // Если это внешняя переменная типа float q
            if (node.Type == typeof(float) || node.Type == typeof(double))
            {
                var objectMember = Expression.Convert(node, typeof(object));
                var getterLambda =
Expression.Lambda<Func<object>>(objectMember);
                float val = Convert.ToSingle(getterLambda.Compile()());


_bodyCode.Append(val.ToString(System.Globalization.CultureInfo.InvariantCulture)
+ "f");
                return node;
            }

            return base.VisitMember(node);
        }

    }
}
```

```
namespace cstorchcl.compiler
{
    public class LocalRoslynTranspiler : CSharpSyntaxRewriter
    {
        private readonly StringBuilder _bodyCode = new StringBuilder();
        private readonly StringBuilder _backwardBodyCode = new StringBuilder();

        public HashSet<string> DiscoveredTensors { get; } = new
HashSet<string>(StringComparer.OrdinalIgnoreCase);
        private Dictionary<string, float> _externalConstants;

        // Словарь для быстрого доступа к метаданным тензоров во время обхода AST
        private Dictionary<string, core.Tensor> _tensorMeta;

        public (string forwardSource, string backwardSource) TranslateWithGrad(
            string pseudoCode,
            Dictionary<string, float> constants,
            Dictionary<string, core.Tensor> tensors)
        {
            _externalConstants = constants ?? new Dictionary<string, float>();
            _tensorMeta = tensors ?? throw new ArgumentNullException(nameof(tensors));

            _bodyCode.Clear();
            _backwardBodyCode.Clear();
            DiscoveredTensors.Clear();
            OutputTensorNames.Clear(); // Очищаем список выходов перед новым проходом
            SyntaxTree tree = CSharpSyntaxTree.ParseText(pseudoCode);
            CompilationUnitSyntax root = tree.GetCompilationUnitRoot();
            Visit(root);

            string utilities = @"// Встроенные утилиты генерации шума и Dropout
inline float nn_rand(int tid, unsigned int seed) {
    unsigned int x = tid + seed;
    x = (x ^ 61) ^ (x >> 16);
    x += (x << 3);
    x = x ^ (x >> 4);
    x *= 0x27d4eb2d;
    x = x ^ (x >> 15);
    return (float)x / 4294967296.0f;
}
inline float nn_dropout(float x, float p, int tid, unsigned int seed) {
    if (p <= 0.0f) return x;
    if (p >= 1.0f) return 0.0f;
    float r = nn_rand(tid, seed);
    return (r < p) ? 0.0f : x / (1.0f - p);
}
inline float nn_dropout_backward(float grad_out, float p, int tid, unsigned int seed) {
    float r = nn_rand(tid, seed);
    return (r < p) ? 0.0f : grad_out / (1.0f - p);
}
```

```
";

        // Вычисляем максимальный ранг (количество осей) среди всех обнаруженных тензоров
        var activeTensors = DiscoveredTensors.Select(name => _tensorMeta[name]).ToList();
        int maxRank = activeTensors.Max(t => t.Shape.Length);

        // За эталонную сетку берем форму самого большого тензора (у которого ранг максимален)
        var maxTensor = activeTensors.First(t => t.Shape.Length == maxRank);
        int[] maxShape = maxTensor.Shape;

        // --- ГЕНЕРАЦИЯ ДЕКОДЕРА КООРДИНАТ (Broadcasting Core) ---
        var coordDecoder = new StringBuilder();
        coordDecoder.Append("    // Восстановление многомерных координат из плоского tid\n");
        coordDecoder.Append($"    int rem = tid;\n");

        // Генерируем переменные координат: dim_0, dim_1...
        for (int i = maxRank - 1; i >= 0; i--)
        {
            if (i > 0)
            {
                coordDecoder.Append($"    int dim_{i} = rem % {maxShape[i]};\n");
                coordDecoder.Append($"    rem /= {maxShape[i]};\n");
            }
            else
            {
                coordDecoder.Append($"    int dim_{i} = rem;\n");
            }
        }
        coordDecoder.Append("\n");

        // --- ГЕНЕРАЦИЯ ИНДЕКСОВ ЧТЕНИЯ ДЛЯ КАЖДОГО ТЕНЗОРА ---
        var indexCalculators = new StringBuilder();
        indexCalculators.Append("    // Вычисление смещений с учетом правил Broadcasting\n");
        foreach (var name in DiscoveredTensors)
        {
            var t = _tensorMeta[name];
            indexCalculators.Append($"    int idx_{name} = 0;\n");

            // Выравниваем оси тензора по правому краю относительно максимального ранга
            int rankDiff = maxRank - t.Shape.Length;

            for (int i = 0; i < t.Shape.Length; i++)
            {
                int targetMaxDimIdx = i + rankDiff;

                // ПРАВИЛО BROADCASTING: если размерность оси равна 1,
                // то индекс по этой оси всегда 0, шаг зануляется (умножение на 0)
                if (t.Shape[i] == 1)
                {
```

```
                indexCalculators.Append($"    idx_{name} += 0 * {t.Strides[i]};\n");
            }
            else
            {
                // Иначе смещаемся на координату эталонной сетки, умноженную на stride оси тензора
                indexCalculators.Append($"    idx_{name} += dim_{targetMaxDimIdx} * {t.Strides[i]};\n");
            }
        }
        // Защита от выхода за физические границы памяти
        indexCalculators.Append($"    idx_{name} = min(idx_{name}, {t.Size} - 1);\n\n");
    }

    // --- СБОРКА СТРОКИ ЯДРА FORWARD ---
    var fSrc = new StringBuilder(utilities);
    fSrc.Append("kernel void fused_elementwise_kernel(\n");
    BuildSignature(fSrc);
    //fSrc.Append("    int tid = get_global_id(0);\n\n");
    fSrc.Append("int tid=get_global_size(1)*get_global_id(0)+get_global_id(1);\n\n");//XY оптимизация
    fSrc.Append(coordDecoder.ToString());
    fSrc.Append(indexCalculators.ToString());

    // Чтение в регистры по вычисленным индексам слияния размерностей
    fSrc.Append("    // Чтение во внутренние регистры\n");
    foreach (var name in DiscoveredTensors)
        fSrc.Append($"    float priv_{name} = {name}[idx_{name}];\n");

    fSrc.Append("\n    // Вычисления\n" + _bodyCode.ToString() + "\n");

    // Запись результатов во VRAM
    fSrc.Append("    // Сохранение результатов\n");
    foreach (var name in DiscoveredTensors)
    {
        fSrc.Append($"    if (tid < {name}_len) {{ {name}[idx_{name}] = priv_{name}; }}\n");
    }
    fSrc.Append("}");

    // --- СБОРКА СТРОКИ ЯДРА BACKWARD ---
    var bSrc = new StringBuilder(utilities);
    bSrc.Append("kernel void fused_elementwise_backward_kernel(\n");
    foreach (var name in DiscoveredTensors)
    {
        bSrc.Append($"    global float* {name}, global float* grad_{name}, global float* out_grad_{name}, const int {name}_len,\n");
    }
    bSrc.Length -= 2;
    bSrc.Append(") {\n");
    //bSrc.Append("    int tid = get_global_id(0);\n\n");
    bSrc.Append("int tid=get_global_size(1)*get_global_id(0)+get_global_id(1);\n\n");//XY оптимизация
```

```
            bSrc.Append(coordDecoder.ToString());
            bSrc.Append(indexCalculators.ToString());

            foreach (var name in DiscoveredTensors)
            {
                bSrc.Append($"    float priv_{name} = {name}[idx_{name}];\n");
                bSrc.Append($"    float priv_grad_{name} = grad_{name}[idx_{name}];\n");
                bSrc.Append($"    float priv_out_grad_{name} = 0.0f;\n");
            }
            bSrc.Append("\n    // Вычисление производных на регистрах\n");
            bSrc.Append(_backwardBodyCode.ToString());
            bSrc.Append("\n    // Сохранение рассчитанных градиентов\n");
            foreach (var name in DiscoveredTensors)
            {
                bSrc.Append($"    if (tid < {name}_len) {{ out_grad_{name}[idx_{name}] =
priv_out_grad_{name}; }}\n");
            }
            bSrc.Append("}");

            return (fSrc.ToString(), bSrc.ToString());
        }

        private void BuildSignature(StringBuilder sb)
        {
            int idx = 0;
            foreach (var name in DiscoveredTensors)
            {
                sb.Append($"    global float* {name}, const int {name}_len");
                if (idx < DiscoveredTensors.Count - 1) sb.Append(",\n");
                idx++;
            }
            sb.Append(") {\n");
        }


        /// <summary>
        /// Список имен тензоров, в которые производилась запись (левая часть оператора '=').
        /// </summary>
        public HashSet<string> OutputTensorNames { get; } = new
HashSet<string>(StringComparer.OrdinalIgnoreCase);


        public override SyntaxNode VisitAssignmentExpression(AssignmentExpressionSyntax node)
        {
            string leftVar = node.Left.ToString().Trim();
            DiscoveredTensors.Add(leftVar);

            // ФИКСАЦИЯ ВЫХОДА: Маркируем тензор как выходной, так как в него идет запись
            OutputTensorNames.Add(leftVar);

            string opStr = node.OperatorToken.Text;
            _bodyCode.Append($"    priv_{leftVar} {opStr} ");
```

```
            ProcessRightSide(node.Right, leftVar);
            _bodyCode.Append(";\n");

            return node;
        }

        private void ProcessRightSide(ExpressionSyntax expression, string leftVar)
        {
            if (expression is InvocationExpressionSyntax invocation)
            {
                // 1. ПЕРЕХВАТ ФУНКЦИЙ (tanh, sin, cos, dropout)
                string funcText = invocation.Expression.ToString();
                var firstArgExpr = invocation.ArgumentList.Arguments[0].Expression;
                string argName = firstArgExpr.ToString().Trim();

                DiscoveredTensors.Add(argName);

                if (funcText.Contains("tanh"))
                {
                    _bodyCode.Append($"tanh(priv_{argName})");
                    _backwardBodyCode.Append($"    priv_out_grad_{argName} += priv_grad_{leftVar}
* (1.0f - priv_{leftVar} * priv_{leftVar});\n");
                }
                else if (funcText.Contains("sin"))
                {
                    _bodyCode.Append($"sin(priv_{argName})");
                    _backwardBodyCode.Append($"    priv_out_grad_{argName} += priv_grad_{leftVar}
* cos(priv_{argName});\n");
                }
                else if (funcText.Contains("cos"))
                {
                    _bodyCode.Append($"cos(priv_{argName})");
                    _backwardBodyCode.Append($"    priv_out_grad_{argName} += priv_grad_{leftVar}
* (-sin(priv_{argName}));\n");
                }
                else if (funcText.Contains("dropout"))
                {
                    string pValStr = invocation.ArgumentList.Arguments[1].ToString();
                    string seedValStr = invocation.ArgumentList.Arguments[2].ToString();

                    _bodyCode.Append($"nn_dropout(priv_{argName}, {pValStr}f, tid, {seedValStr}u)");
                    _backwardBodyCode.Append($"    priv_out_grad_{argName} +=
nn_dropout_backward(priv_grad_{leftVar}, {pValStr}f, tid, {seedValStr}u);\n");
                }
            }
            else if (expression is BinaryExpressionSyntax binary)
            {
                // 2. ПЕРЕХВАТ БИНАРНОЙ МАТЕМАТИКИ (+, -, *, /)
                // Исправление 1: Рекурсивно раскрываем операнды, знаки подчеркивания
добавятся внутри IdentifierNameSyntax автоматически!
                ProcessRightSide(binary.Left, leftVar);
```

```
            string op = binary.OperatorToken.Text;
            _bodyCode.Append($" {op} ");

            ProcessRightSide(binary.Right, leftVar);

            string leftStr = binary.Left.ToString().Trim();
            string rightStr = binary.Right.ToString().Trim();

            if (op == "/" && _externalConstants.TryGetValue(rightStr, out float val))
            {
                string valStr = val.ToString("F1", System.Globalization.CultureInfo.InvariantCulture)
+ "f";
                _backwardBodyCode.Append($"    priv_out_grad_{leftStr} += priv_grad_{leftVar} /
{valStr};\n");
            }
            else if (op == "+" && !_externalConstants.ContainsKey(leftStr) && !
_externalConstants.ContainsKey(rightStr))
            {
                // Исправление 2: Добавлены потерянные знаки подчеркивания '_' для обратного
прохода сложения тензоров
                if (DiscoveredTensors.Contains(leftStr))
                    _backwardBodyCode.Append($"    priv_out_grad_{leftStr} +=
priv_grad_{leftVar};\n");

                if (DiscoveredTensors.Contains(rightStr))
                    _backwardBodyCode.Append($"    priv_out_grad_{rightStr} +=
priv_grad_{leftVar};\n");
            }
        }
        else if (expression is IdentifierNameSyntax identifier)
        {
            // 3. ПЕРЕХВАТ ИДЕНТИФИКАТОРОВ (Переменные-тензоры)
            string name = identifier.Identifier.Text.Trim();

            if (_externalConstants.TryGetValue(name, out float val))
            {
                _bodyCode.Append(val.ToString("F1",
System.Globalization.CultureInfo.InvariantCulture) + "f");
            }
            else
            {
                DiscoveredTensors.Add(name);
                // Исправление 3: Строго добавляем префикс со знаком подчеркивания для
прямого прохода
                _bodyCode.Append($"priv_{name}");
            }
        }
        else
        {
            // 4. ПРЯМЫЕ ЧИСЛОВЫЕ КОНСТАНТЫ ИЗ ТЕКСТА
            string numStr = expression.ToString().Trim().Replace(",", ".");
            if (!numStr.Contains(".")) numStr += ".0";
```

```
            if (!numStr.EndsWith("f") && !numStr.EndsWith("u")) numStr += "f";
            _bodyCode.Append(numStr);
        }
    }

}
}
```

```
using System;
using cstorchcl.core;

namespace cstorchcl.compiler
{
    public class LocalVariable
    {
        public Tensor Tensor { get; }
        public string Name { get; }

        // Передаем метаданные формы для анализа Broadcasting
        public int[] Shape => Tensor.Shape;
        public int[] Strides => Tensor.Strides;

        public LocalVariable(Tensor tensor, string name)
        {
            Tensor = tensor ?? throw new ArgumentNullException(nameof(tensor));
            Name = name;
        }

        /// <summary>
        /// Эквивалент оператора присваивания (=) для обхода ограничений
Expression Trees.
        /// </summary>
        public LocalVariable Assign(LocalVariable value) => this;
        public LocalVariable Assign(double value) => this;

        // Перегруженные операторы теперь просто участвуют в построении
математического дерева
        public static LocalVariable operator +(LocalVariable a, double b) => a;
        public static LocalVariable operator -(LocalVariable a, double b) => a;
        public static LocalVariable operator *(LocalVariable a, double b) => a;
        public static LocalVariable operator /(LocalVariable a, double b) => a;

        public static LocalVariable operator +(LocalVariable a, LocalVariable b)
=> a;
        public static LocalVariable operator -(LocalVariable a, LocalVariable b)
=> a;
        public static LocalVariable operator *(LocalVariable a, LocalVariable b)
=> a;
        public static LocalVariable operator /(LocalVariable a, LocalVariable b)
=> a;
    }

    public static class nnclloc
    {
        public static LocalVariable sin(LocalVariable v) => v;
        public static LocalVariable cos(LocalVariable v) => v;
        public static LocalVariable tanh(LocalVariable v) => v;
        public static LocalVariable dropout(LocalVariable v, float p, uint seed)
=> v;
    }
}
```

```
namespace cstorchcl.compiler
{
    public static class NnClLoc
    {
        /// <summary>
        /// Выполняет динамическую сборку и запуск сплавленного OpenCL-ядра на
2D-сетке варпов.
        /// </summary>
        ///         /// <summary>
        /// Выполняет динамическую сборку и возвращает СЛОВАРЬ новых выходных
тензоров.
        /// </summary>
        public static Dictionary<string, Tensor> Run(string pseudoCode,
Dictionary<string, Tensor> tensors, ComputeContext computeCtx,
Dictionary<string, float> constants = null)
        {
            if (string.IsNullOrWhiteSpace(pseudoCode)) return null;
            if (tensors == null || tensors.Count == 0) return null;

            var transpiler = new LocalRoslynTranspiler();
            var (forwardSource, backwardSource) =
transpiler.TranslateWithGrad(pseudoCode, constants, tensors);

            int N = tensors.Values.Max(t => t.Size);
            CommandQueue queue = DeviceQueueRegistry.GetCurrent();
            Context context = queue.Context;

            //
============================================================================
            // ИСПРАВЛЕНИЕ: Выделяем новые изолированные буферы для результатов
вычислений
            //
============================================================================
            var outputDict = new Dictionary<string,
Tensor>(StringComparer.OrdinalIgnoreCase);

            foreach (var outName in transpiler.OutputTensorNames)
            {
                Tensor originalIn = tensors[outName];

                // Создаем НОВЫЙ тензор. Он будет чистым результатом JIT-блока!
                Tensor newOut = new Tensor(originalIn.Shape, context,
MemoryFlags.ReadWrite,
                    requiresGrad: tensors.Values.Any(t => t.RequiresGrad));

                outputDict.Add(outName, newOut);
            }

            // Создаем рабочий словарь для запуска ядра на GPU.
            // Заменяем выходные буферы на новые чистые структуры, чтобы не
затирать входы!
            var executionTensors = new Dictionary<string, Tensor>(tensors,
StringComparer.OrdinalIgnoreCase);
            foreach (var kvp in outputDict)
            {
                executionTensors[kvp.Key] = kvp.Value;
            }
            //
============================================================================

            int warp = computeCtx.warp_size;
```

```
            int globalX = ((1 + (N / warp)) + computeCtx.MAX_WORK_GPOUP_SIZE_X -
1) / computeCtx.MAX_WORK_GPOUP_SIZE_X * computeCtx.MAX_WORK_GPOUP_SIZE_X;
            int globalY = ((warp) + computeCtx.MAX_WORK_GPOUP_SIZE_Y - 1) /
computeCtx.MAX_WORK_GPOUP_SIZE_Y * computeCtx.MAX_WORK_GPOUP_SIZE_Y;

            Program program = JITProgramCache.GetOrCompile(context,
queue.Device, forwardSource);
            using Kernel kernel =
program.CreateKernel("fused_elementwise_kernel");

            int argIndex = 0;
            foreach (var tensorName in transpiler.DiscoveredTensors)
            {
                Tensor t = executionTensors[tensorName]; // Читает из старых,
пишет в новые!
                kernel.Arguments[argIndex++].SetValue(t.DataBuffer);
                kernel.Arguments[argIndex++].SetValue(t.Size);
            }

            queue.EnqueueNDRangeKernel(kernel, null, new[] { new
IntPtr(globalX), new IntPtr(globalY) }, new[] { new
IntPtr(computeCtx.MAX_WORK_GPOUP_SIZE_X), new
IntPtr(computeCtx.MAX_WORK_GPOUP_SIZE_Y) });
            queue.Finish();

            // Интеграция с Autograd
            bool anyRequiresGrad = tensors.Values.Any(t => t.RequiresGrad);
            // ИСПРАВЛЕНИЕ ИНТЕГРАЦИИ: Убираем дублирование слоев в Autograd

            if (anyRequiresGrad)
            {
                Tensor[] inputs = tensors.Values.ToArray();
                Tensor representative = outputDict.Values.First();

                var localOp = new LocalOp(inputs, representative,
backwardSource, executionTensors, computeCtx);

                // Назначаем Creator СТРОГО только выходным тензорам JIT-блока!
                // Входной projected сохранит свой BmmOp, а Bias останется
листом графа (Creator = null)
                foreach (var outTensor in outputDict.Values)
                {
                    outTensor.Creator = localOp;
                }
            }


            return outputDict; // Возвращаем коллекцию новых результатов
        }


    }
}
```

```
namespace cstorchcl.core
{
    /// <summary>
    /// Динамический узел графа автоматического дифференцирования для JIT-
сплавленных ядер LocalRunner.
    /// </summary>
    public class LocalOp : Op
    {
        private readonly string _backwardKernelSource;
        private readonly Dictionary<string, Tensor> _tensorArgs;

        // Добавьте поле в класс LocalOp:
        private readonly ComputeContext _computeCtx;

        // Обновите конструктор класса LocalOp, чтобы он принимал
ComputeContext:
        public LocalOp(Tensor[] inputs, Tensor output, string backwardSource,
Dictionary<string, Tensor> tensorArgs, ComputeContext computeCtx)
        {
            Inputs = inputs;
            Output = output;
            _backwardKernelSource = backwardSource;
            _tensorArgs = tensorArgs;
            _computeCtx = computeCtx;
        }

        public override Tensor[] Backward(Tensor gradOutput, Context context)
        {
            CommandQueue queue = DeviceQueueRegistry.GetCurrent();
#if DEBUG
            Console.WriteLine("[locRunner]запуск обратного прохода!");
#endif
            var prg = compiler.JITProgramCache.GetOrCompile(context,
queue.Device, _backwardKernelSource);
            using Kernel kernel =
prg.CreateKernel("fused_elementwise_backward_kernel");

            var outputGrads = new Dictionary<string, Tensor>();
            int argIndex = 0;

            foreach (var kvp in _tensorArgs)
            {
                string name = kvp.Key;
                Tensor t = kvp.Value;

                t.InitGradient(context);

                // Выделяем временный буфер для дельты текущего слоя
                Tensor outLayerGrad = new Tensor(t.Shape, context,
MemoryFlags.ReadWrite, requiresGrad: false);
                outputGrads.Add(name, outLayerGrad);

                //
================================================================================
                // ИСПРАВЛЕНИЕ СЦЕПКИ ГРАДИЕНТОВ:
                // Если этот тензор в executionTensors является выходом
(например, "P"),
                // то его входящим градиентом для JIT-математики должен быть
gradOutput,
                // который прислал верхний слой (Softmax), а не его собственный
внутренний t.Grad!
```

```
                //
================================================================================
                NOpenCL.Buffer incomingGradBuffer = t.Grad.DataBuffer;

                // Проверяем, совпадает ли буфер с буфером репрезентативного
выхода операции
                if (t.DataBuffer == Output.DataBuffer)
                {
                    incomingGradBuffer = gradOutput.DataBuffer; // Подменяем
градиент на пришедший сверху!
                }
                //
================================================================================

                // Передаем аргументы в Си-ядро OpenCL
                kernel.Arguments[argIndex++].SetValue(t.DataBuffer);
                kernel.Arguments[argIndex++].SetValue(incomingGradBuffer); //
Используем правильный входящий поток градиента!
                kernel.Arguments[argIndex++].SetValue(outLayerGrad.DataBuffer);
                kernel.Arguments[argIndex++].SetValue(t.Size);
            }

            // Нахождение максимального размера сетки потоков
            int maxTensorSize = _tensorArgs.Values.Max(t => t.Size);

            int warp = _computeCtx.warp_size;
            //
================================================================================
            // ИСПРАВЛЕНИЕ: Точное выравнивание 2D варп-сетки для обратного
прохода
            //
================================================================================
            //int maxTensorSize = _tensorArgs.Values.Max(t => t.Size);


            int localX = _computeCtx.MAX_WORK_GPOUP_SIZE_X; // 256
            int localY = _computeCtx.MAX_WORK_GPOUP_SIZE_Y; // 4

            // Вычисляем глобальные размеры строго по вашей формуле
            int rawGlobalY = warp;
            int rawGlobalX = 1 + (maxTensorSize / rawGlobalY);

            // Округляем вверх до кратности локальным группам
            int globalX = ((rawGlobalX + localX - 1) / localX) * localX;
            int globalY = ((rawGlobalY + localY - 1) / localY) * localY;

            IntPtr[] globalWorkSize = new[] { new IntPtr(globalX), new
IntPtr(globalY) };
            IntPtr[] localWorkSize = new[] { new IntPtr(localX), new
IntPtr(localY) }; // Передаем X и Y без путаницы!
            //
================================================================================

            queue.EnqueueNDRangeKernel(kernel, null, globalWorkSize,
localWorkSize);
            queue.Finish();

            queue.Finish();

            // Формируем результирующий массив градиентов для возврата в
AutoGrad
            // ... (после выполнения ядра queue.Finish(); на странице 3)
```

```
            // Формируем чистый результирующий массив изолированных градиентов
для возврата в AutoGrad
            Tensor[] finalResultGrads = new Tensor[Inputs.Length];

            for (int i = 0; i < Inputs.Length; i++)
            {
                Tensor inputTensor = Inputs[i];
                Tensor calculatedGradForLayer = null;

                // Находим, под каким строковым именем этот входной тензор шел в
JIT-блоке
                foreach (var kvp in _tensorArgs)
                {
                    if (kvp.Value == inputTensor)
                    {
                        // Извлекаем буфер градиента, который посчитало наше Си-
ядро
                        calculatedGradForLayer = outputGrads[kvp.Key];
                        break;
                    }
                }

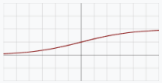
                if (calculatedGradForLayer != null)
                {
                    // Важный шаг (как у BmmOp): сбрасываем создателя, чтобы
этот тензор градиента
                    // считался чистыми данными и не участвовал в обходе графа
                    calculatedGradForLayer.Creator = null;
                    finalResultGrads[i] = calculatedGradForLayer;
                }
                else
                {
                    finalResultGrads[i] = null;
                }
            }

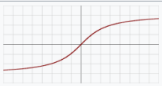
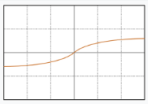
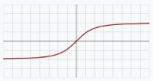
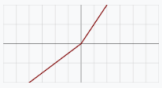
            // Возвращаем изолированные градиенты. Движок AutoGrad сам сложит
их!
            return finalResultGrads;
        }

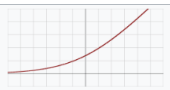
    }

}
```

Теперь каксаемо реализаций функций активации:

Логистическая (сигмоида или Гладкая ступенька)		$f(x) = \sigma(x) = \frac{1}{1+e^{-x}}$[1]	$f'(x) = f(x)(1-f(x))$

arctg		$f(x) = \operatorname{tg}^{-1}(x)$	$f'(x) = \frac{1}{x^2+1}$
Softsign[9][10]		$f(x) = \frac{x}{1+\lvert x\rvert}$	$f'(x) = \frac{1}{(1+\lvert x\rvert)^2}$
Обратный квадратный корень (англ. *Inverse square root unit*, ISRU)[11]		$f(x) = \frac{x}{\sqrt{1+\alpha x^2}}$	$f'(x) = \left(\frac{1}{\sqrt{1+\alpha x^2}}\right)^3$
Линейный выпрямитель[англ.] (или Полулинейный элемент) (англ. *Rectified linear unit*, ReLU) [12][13]		$f(x) = \begin{cases} 0 & x < 0 \\ x & x \geqslant 0 \end{cases}$	$f'(x) = \begin{cases} 0 & x < 0 \\ 1 & x \geqslant 0 \end{cases}$
Линейный выпрямитель с «утечкой» (англ. *Leaky rectified linear unit*, Leaky ReLU)[14]		$f(x) = \begin{cases} 0,01x & x < 0 \\ x & x \geqslant 0 \end{cases}$	$f'(x) = \begin{cases} 0,01 & x < 0 \\ 1 & x \geqslant 0 \end{cases}$

SoftPlus[21]		$f(x) = \ln(1+e^x)$	$f'(x) = \frac{1}{1+e^{-x}}$
Выгнутая тождественная функция (англ. *Bent identity*)		$f(x) = \frac{\sqrt{x^2+1}-1}{2} + x$	$f'(x) = \frac{x}{2\sqrt{x^2+1}} + 1$
Сигмоидно-взвешенная линейная функция (англ. *Sigmoid-weighted linear unit*, SiLU)[22]		$f(x) = x \cdot \sigma(x)$[5]	$f'(x) = f(x) + \sigma(x)(1-f(x))$[6]

На мой взгляд имеет смысл реализовать данные функции как и droput как локальные cl функции.

```
        // Встраиваем ваш быстрый генератор случайных чисел и логику дропаута в начало кода
        source.Append(@"// Встроенные утилиты генерации шума и Dropout
inline float nn_rand(int tid, unsigned int seed) {
    unsigned int x = tid + seed;
    x = (x ^ 61) ^ (x >> 16);
    x += (x << 3);
    x = x ^ (x >> 4);
    x *= 0x27d4eb2d;
    x = x ^ (x >> 15);
    return (float)x / 4294967296.0f;
}
inline float nn_dropout(float x, float p, int tid, unsigned int seed) {
    if (p <= 0.0f) return x;
    if (p >= 1.0f) return 0.0f;
    float r = nn_rand(tid, seed);
    if (r < p) {
        return 0.0f;
    } else {
        return x / (1.0f - p);
    }
}

");
```

Аналогично для обратного прохода:

```
        OutputTensorNames.Clear(); // Очищаем список выходов перед новым проходом
        SyntaxTree tree = CSharpSyntaxTree.ParseText(pseudoCode);
        CompilationUnitSyntax root = tree.GetCompilationUnitRoot();
        Visit(root);

        string utilities = @"// Встроенные утилиты генерации шума и Dropout
inline float nn_rand(int tid, unsigned int seed) {
    unsigned int x = tid + seed;
    x = (x ^ 61) ^ (x >> 16);
    x += (x << 3);
    x = x ^ (x >> 4);
    x *= 0x27d4eb2d;
    x = x ^ (x >> 15);
    return (float)x / 4294967296.0f;
}
inline float nn_dropout(float x, float p, int tid, unsigned int seed) {
    if (p <= 0.0f) return x;
    if (p >= 1.0f) return 0.0f;
    float r = nn_rand(tid, seed);
    return (r < p) ? 0.0f : x / (1.0f - p);
}
inline float nn_dropout_backward(float grad_out, float p, int tid, unsigned int seed) {
    float r = nn_rand(tid, seed);
    return (r < p) ? 0.0f : grad_out / (1.0f - p);
}

";
```

Тогда сильно упроститься следующий код:

```
    }
    else if (funcText.Contains("cos"))
    {
        _bodyCode.Append($"cos(priv_{argName})");
        _backwardBodyCode.Append($"    priv_out_grad_{argName} += priv_grad_{leftVar} * (-sin(priv_{argName}));\n");
    }
    else if (funcText.Contains("dropout"))
    {
        string pValStr = invocation.ArgumentList.Arguments[1].ToString();
        string seedValStr = invocation.ArgumentList.Arguments[2].ToString();

        _bodyCode.Append($"nn_dropout(priv_{argName}, {pValStr}f, tid, {seedValStr}u)");
        _backwardBodyCode.Append($"    priv_out_grad_{argName} += nn_dropout_backward(priv_grad_{leftVar}, {pValStr}f, tid, {seedValStr}u);\n");
    }
```

Можно будет прогнать строку через конвеер :
f_names=[“relu”,”softplus” ,…];
foreach(fn in f_names)
if(funcText.contains(fn)
{
_bodyCode.Append($”{fn}(priv_{argName})”);
_backwardBodyCode.Append($"   priv_out_grad_{argName} += priv_grad_{leftVar} * ({fn}_backward(priv_{argName}));\n");

}